



Programmieren

Falk Morgenstern
Leon Johann Brettin

Foliensatz basierend auf Unterlagen von Prof. Huhn, Prof. Meyer und weiteren.
Nur zur Verwendung in der Vorlesung „Programmieren“ an der Ostfalia.
Keine Weitergabe.



Programmieren

12. Lambdas und Streams



Programmieren

12. Lambdas und Streams

12.1 Lambdas



λ-Ausdrücke vs. Innere Klassen in Event-Handlern

- Lambda-Ausdrücke erlauben es, im Code „nur das Wesentliche zu schreiben, ohne dabei jedes Mal das Übliche wiederholen zu müssen“.
- Java 7 und davor: Event-Handler mit anonymer innerer Klasse

```
button.setAction(new EventHandler<ActionEvent>() {  
    @Override  
    public void handle(ActionEvent e) {  
        doSomethingWith(e);  
    }  
});
```



- Seit Java 8: λ-Ausdrücke

```
button.setAction(e -> doSomethingWith(e));
```





λ-Ausdrücke vs. Innere Klassen als Comparator

- Java 7 und davor: Event-Handler mit anonymer innerer Klasse

```
String[] sarr = {"abc", "adef"};

Arrays.sort(sarr, new Comparator<String>() {
    @Override
    public int compare(String s1, String s2) {
        return s1.length() - s2.length();
    }
});
```



- Seit Java 8: λ-Ausdrücke

```
Arrays.sort(sarr, (s1, s2) -> s1.length() - s2.length());
```





λ -Ausdrücke in funktionalen Interfaces

- λ -Ausdrücke sehen aus wie eine Funktion

```
button.setOnAction(e -> doSomethingWith(e));
```



```
Arrays.sort(sarr, (s1,s2) -> s1.length() - s2.length());
```



- Der Compiler erkennt, dass eine Instanz einer Klasse benötigt wird, die das Interface `EventHandler<ActionEvent>` bzw. `Comparator<String>` implementiert, und erzeugt sie.
- Wie das?
- Lösung: **Functional Interface = single, abstract method interface (SAM)**



Funktionale Interfaces

- Ein **Functional Interface** in Java ist ein Interface, das eine einzige nicht-implementierte Methode enthält.
- Functional interfaces können über λ -Ausdrücke implementiert werden.
- Das Interface darf weitere default-Methoden (also mit Implementierung) enthalten
- Beispiel Arrays: `sort (T[] a, Comparator<? super T> c)`
 - Der Compiler weiß, dass eine Implementierung von `Comparator` erwartet wird, die den Methodenrumpf von `compare` mitbringt.

```
Arrays.sort(sarr, (s1,s2) -> s1.length() - s2.length());
```





λ-Syntax

- Dem Compiler ist bekannt
 - es folgt eine Implementierung eines Comparators
 - der Name der zu implementierenden Methode
 - konkreter Typ für den Typparameter T (hier: String)
 - Wenn der Rumpf nur aus einem Ausdruck besteht genügt der Ausdruck

```
Arrays.sort(sarr, new Comparator<String>() {  
    @Override  
    public int compare(String s1, String s2) {  
        return s1.length() - s2.length();  
    }  
});
```



```
Arrays.sort(sarr, (s1, s2) -> s1.length() - s2.length());
```





λ-Syntax

- Hinweise:
 - Falls der Compiler den Typ nicht selbst bestimmen kann, wird gewarnt, dann Typ mit angeben
 - Falls der Methodenrumpf komplexer ist, kann auch ein Block mit einem `return` verwendet werden (gewollt ist aber ein „einfacher“ λ-Ausdruck)
 - Wenn die Methode genau einen Parameter bekommt, dürfen auch die runden Klammern um die Parameter entfallen.

```
Thread t = new Thread(() -> foo()); // kein Parameter, Klammern notwendig
```



```
Arrays.sort(sarr, (s1,s2) -> s1.length() - s2.length()); // mehr als ein  
// Parameter, Klammern notwendig
```





λ -Syntax

- Funktionaler Stil mit λ -Ausdruck:
- ersetzt Nennung des funktionalen Interfaces, der Methode, der Parametertypen,... und erlaubt lesbare, kompakte Programmierung.
- Aber das ist **Syntactic Sugar**: Der Java-Compiler behandelt den λ -Ausdruck intern als Abkürzung für die Erzeugung einer Instanz einer anonymen Klasse.
- Ein Lambda-Ausdruck ist zuweisungskompatibel zu jedem Funktionsinterface, dessen abstrakte Methode die passende Parameterliste und den passenden Rückgabetyp hat.

```
Arrays.sort(sarr, (s1,s2) -> s1.length() - s2.length());
```





Beispiele für λ -Ausdrücke

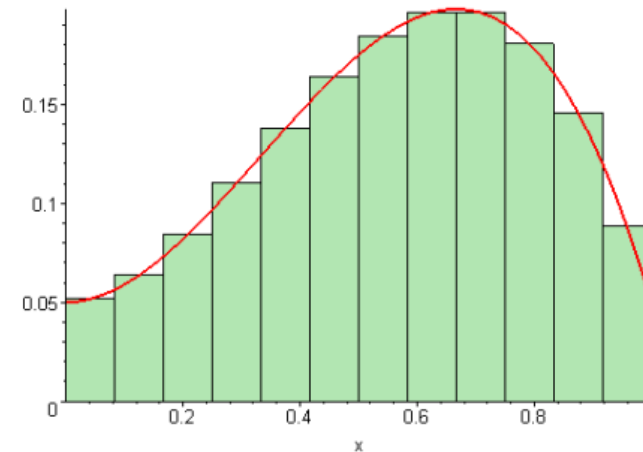
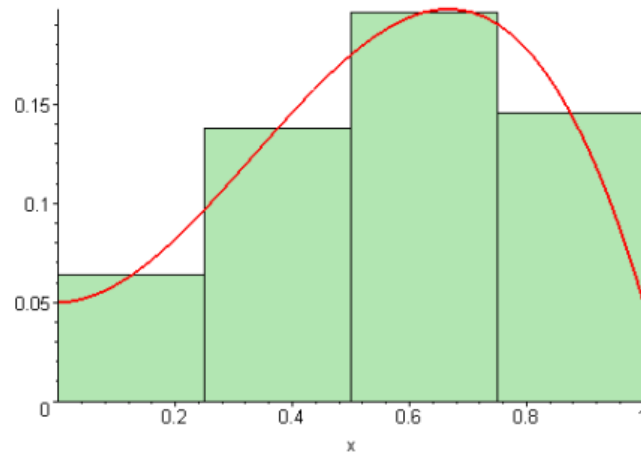
```
x -> x*x  
(x,y) -> x - y  
(x,y) -> {return x - y;}  
(s1, s2) -> System.out.println(s1 + s2)  
( ) -> 42
```



- Beispiel:
 - Das funktionale Interface `java.util.Comparator<T>` definiert die abstrakte Methode `int compare(T obj1, T obj2)`. Sie vergleicht `obj1` und `obj2` und liefert ein Ergebnis kleiner Null / Null / größer Null, je nachdem, ob `obj1` kleiner / gleich / größer als `obj2` ist.
 - Der λ -Ausdruck `(x, y) -> x - y` ist zuweisungskompatibel zu `Comparator<Integer>`, denn zwei Zahlen als Parameter liefern eine Zahl als Ergebnis.



Beispiele für λ -Ausdrücke: Numerische Integration



Quelle: Wikimedia

- Numerische Integration über Riemannsche Summen implementieren
- Benötigt werden die Grenzen, die Intervallbreite, und ein Funktionswert $f(x_i)$ pro Rechteck R_i
- Die zu integrierende Funktion f wird als λ -Ausdruck übergeben



Functional Interface: Integrable

```
public interface Integrable {  
  
    double eval(double x); // auszuwertende Funktion  
  
    public static double integrate(Integrable function,  
                                   double x1, double x2, int numSlices) {  
        if (numSlices < 1) {  
            numSlices = 1;  
        }  
        double delta = (x2 - x1) / numSlices;  
        double start = x1 + delta / 2;  
        double sum = 0;  
        for (int i = 0; i < numSlices; i++) {  
            sum += delta * function.eval(start + delta * i);  
        }  
        return sum;  
    }  
}
```





Interface Integrable: Auszuwertende Funktion

- Die Implementierung von `eval()` wird die zu integrierende Funktion $x \rightarrow f(x)$
- Beispiele:
 - `x -> x*x`
 - `x -> Math.sin(x) + 1`
 - `x -> Math.pow(x,3)`



Functional Interface: Testtreiber

```
public class NumIntegrationTest {  
    public static void main(String[] args) {  
        integrationOutput(x -> x * x, 1, 5, "f(x) = x*x");  
        integrationOutput(x -> Math.sin(x) + 1, 0, Math.PI, "f(x)=sin(x)+1");  
        integrationOutput(x -> Math.pow(x, 3), 0, 5, "f(x) = x^3");  
    }  
  
    public static String integrationTest(Integrable fct, double x1, double x2) {  
        String output = "";  
        for (int i = 1; i < 7; i++) {  
            int nSlices = (int) Math.pow(10, i);  
            double result = Integrable.integrate(fct, x1, x2, nSlices);  
            output = output + " for " + String.format("%10d", nSlices) + " slices:"  
                + String.format("%.8f", result) + "\n";  
        }  
        return output;  
    }  
  
    public static void integrationOutput(Integrable function, double x1, double x2, String fun_text) {  
        System.out.println("Approximated integral of " + fun_text + " from " + x1 + " to " + x2);  
        System.out.println(integrationTest(function, x1, x2));  
    }  
}
```





Functional Interface Integrable: Ergebnisse

Approximated integral of $f(x) = x*x$ from 1.0 to 5.0

```
for      10 slices: 41,28000000
for     100 slices: 41,33280000
for    1000 slices: 41,33332800
for   10000 slices: 41,33333328
for  100000 slices: 41,33333333
for 1000000 slices: 41,33333333
```

Approximated integral of $f(x)=\sin(x)+1$ from 0.0 to 3.141592653589793

```
for      10 slices: 5,14984106
for     100 slices: 5,14167490
for    1000 slices: 5,14159348
for   10000 slices: 5,14159266
for  100000 slices: 5,14159265
for 1000000 slices: 5,14159265
```

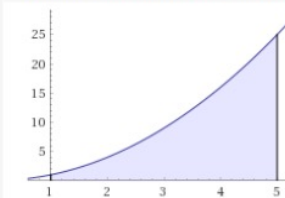
Approximated integral of $f(x) = x^3$ from 0.0 to 5.0

```
for      10 slices: 155,46875000
for     100 slices: 156,24218750
for    1000 slices: 156,24992187
for   10000 slices: 156,24999922
for  100000 slices: 156,24999999
for 1000000 slices: 156,25000000
```

Definite integral:

$$\int_1^5 x x dx = \frac{124}{3} \approx 41.333$$

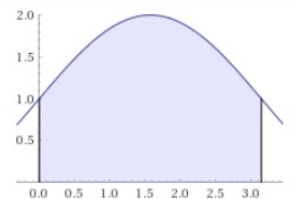
Visual representation of the integral:



Definite integral:

$$\int_0^\pi (\sin(x) + 1) dx = 2 + \pi \approx 5.1416$$

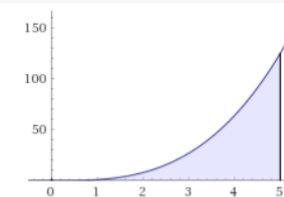
Visual representation of the integral:



Definite integral:

$$\int_0^5 x^3 dx = \frac{625}{4} = 156.25$$

Visual representation of the integral:





Functional Interfaces - Annotation

- Annotation `@FunctionalInterface`
- Jede Schnittstelle mit genau einer abstrakten Methode ist ein **Java Functional Interface** und eignet sich somit für einen λ -Ausdruck.
- Optionale Annotation `@FunctionalInterface` zeigt an, dass das gewollt ist und die Schnittstelle nicht später um weitere abstrakte Methoden erweitert werden soll.
- Compiler prüft, ob `@FunctionalInterface` SAM-Interface ist.

```
@FunctionalInterface  
public interface Integrable {
```





Functional Interfaces - Beispiele

- Es gibt eine Reihe weiterer Functional Interfaces
 - `Predicate<T>` mit `test` testet einen Parameter und gibt `boolean` zurück.
 - `Comparator<T>` mit `compare` vergleicht 2 Objekte und liefert `int` zurück.
 - `BinaryOperator<T>` mit `apply` (2 Parameter) verknüpft zwei Parameter.
 - `UnaryOperator<T>` mit `apply` (1 Parameter) wendet eine Operation auf einen Parameter an
 - `EventHandler<T extends Event>` mit `handle`, das 1 Parameter (Event) erwartet und `void` zurückgibt
 - usw.



Functional Interfaces

- `Comparable` ist zwar syntaktisch ein funktionales Interface, ist aber eher gedacht, um in einer Klasse implementiert zu werden.
- `Runnable` (Interface für die Erzeugung von Threads) mit `run` (ohne Parameter, gibt `void` zurück)
- Weitere Beispiele finden Sie im Package `java.util.function`
<https://docs.oracle.com/javase/8/docs/api/java/util/function/package-summary.html>
- Ein λ -Ausdruck kann in verschiedenen Kontexten genutzt werden. Die Parametertypen werden aus dem Zuweisungskontext ermittelt: **Typ-Inferenz**.



Beispiel BinaryOperator

```
public class BinOpTest {  
    public static <T> T fold(BinaryOperator<T> op, T first, T... items) {  
        T result = first;  
        for (T item : items) { result = op.apply(result, item); }  
        return result;  
    }  
    public static void main(String[] args) {  
        // als funktionales Interface mit anonymer innerer Klasse  
        String s1 = fold(new BinaryOperator<String>() {  
            @Override  
            public String apply(String s1, String s2) { return s1 + s2; }  
        }, "", "a", "b", "c");  
        System.out.println(s1); // abc  
  
        // alternativ als Lambda-Ausdruck, zunächst für String ...  
        String s2 = fold((a, b) -> a + b, "", "a", "b", "c");  
        System.out.println(s2); // abc  
        // und dann für int  
        int x = fold((a, b) -> a + b, 0, 1, 2, 3);  
        System.out.println(x); // 6  
    }  
}
```





Weitere Beispiel aus java.util.function

```
void otherExamples() {  
    Comparator<Integer> comparator = (x, y) -> x - y;  
    //int compare(T obj1, T obj2), i.e. int compare(Integer x, Integer y)  
  
    Predicate<Integer> p = n -> n % 2 == 0;  
    //boolean test(T t), i.e. boolean test(Integer n)  
  
    Function<Integer, Integer> f = x -> x * x;  
    //R apply(T t), i.e. Integer apply(Integer x)  
  
    BinaryOperator<Integer> op = (x, y) -> x + y;  
    //T apply(T t, T u), i.e. Integer apply(Integer x, Integer y)  
  
    Consumer<String> c = s -> System.out.println(s);  
    //void accept(T t), i.e. void accept(String s)  
  
    Supplier<Integer> s = () -> 47411;  
    //T get(), i.e. Integer get()  
}
```





Methodenreferenzen

- Es geht noch einfacher, wenn der λ -Ausdruck nur aus dem Aufruf einer Methode besteht
- Methodenreferenz. Hierbei werden 4 Fälle unterschieden:
 1. Statische Methoden:
`(param) -> Typ.statischeMethode(param)`
`Typ::statischeMethode`
 2. Instanz-Methoden für ein bestimmtes Objekt `ref`:
`(param) -> ref.methode(param)`
`ref::methode`
 3. Instanz-Methoden für Objekt `obj` eines bestimmten Typs `Typ`:
`(obj, param) -> obj.methode(param)`
`Typ::methode`
 4. Konstruktor:
`(param) -> new Klasse(param)`
`Klasse::new`



- Bitte evaluieren Sie jetzt



Methodenreferenzen

- Eine Methodenreferenz ersetzt einen entsprechenden λ -Ausdruck
- Die Signatur der Methode muss zu dem Functional Interface passen, für das die Methode die Implementierung sein soll.
- Die Methodenreferenz ist semantisch äquivalent zum entsprechenden λ -Ausdruck
- Beispiel
 - Formale Parameterliste folgt aus `Comparator<Person>.compare: (Person, Person)`
 - Funktionskörper ruft die Methode `Person.compareByAge` auf.

```
public class Person {  
    LocalDate birthday;  
    public LocalDate getBirthday() {  
        return birthday;  
    }  
  
    public static int compareByAge(Person a,  
                                   Person b) {  
        return a.birthday.compareTo(b.birthday);  
    }  
  
    public static void main(String[] args) {  
        Person[] pA = new Person[3];  
        // ... Objekte erzeugen  
  
        Arrays.sort(pA,  
                    (Person a, Person b) ->  
                    Person.compareByAge(a,b) ); //  $\lambda$   
        Arrays.sort(pA, Person::compareByAge);  
        // Methodenreferenz (Typ 1.)  
    }  
}
```




Methodenreferenzen

- Weiteres Beispiel
`integrationTest(Math::sin, 0, Math.PI)`
- Was ist der Typ von `Math::sin`? Antwort: `double`, `Function`, `Math`?
Falsche Frage!
- Richtige Frage:
Was ist der Typ von `Math::sin` in `integrationTest(...)`?
- Richtige Antwort:
`Math::sin` hat in unserem obigen Beispiel den Typ `Integrable`
- Aber:
In einem anderen Kontext kann die Methodenreferenz `Math::sin` einen anderen Typ haben. Das gilt analog für λ -Ausdrücke.
- Der Typ von Methodenreferenzen und von λ -Ausdrücken ist kontextabhängig.



Beispiele Methodenreferenzen

```
@FunctionalInterface
public interface StringFunction {

    String applyFunction(String s);

    public static String modify(String s, StringFunction f) {
        return f.applyFunction(s);
    }
}
```





Beispiele Methodenreferenzen

```
public class StringTester {  
  
    public static String makeExciting(String s) { return (s + "! Wow!"); }  
  
    public static void main(String[] args) {  
        String s = "Hallo zusammen!";  
        String prefix = " BlaBla...";  
        // Class::classMethod  
        String result1 = StringFunction.modify(s, StringTester::makeExciting);  
        // instance::instanceMethod  
        String result2 = StringFunction.modify(prefix, s::concat);  
        // Class::instanceMethod  
        String result3 = StringFunction.modify(s, String::toUpperCase);  
        System.out.println(result1 + "\n" + result2 + "\n" + result3);  
    }  
}
```





Beispiele Methodenreferenzen

- Ausgabe

```
Hallo zusammen!! Wow!  
Hallo zusammen! BlaBla...  
HALLO ZUSAMMEN!
```

- Vergleich Methodenreferenz / λ -Ausdruck

```
// Class::classMethod, statische Methode, s wird Parameter  
String result1 = StringFunction.modify(s, StringTester::makeExciting);  
String r1a = StringFunction.modify(s, a -> StringTester.makeExciting(a));  
// instance::instanceMethod, Instanz ist s, prefix wird Parameter  
String result2 = StringFunction.modify(prefix, s::concat);  
String r2a = StringFunction.modify(prefix, a -> s.concat(a));  
// Class::instanceMethod, Instanz ist Parameter, s wird Parameter  
String result3 = StringFunction.modify(s, String::toUpperCase);  
String r3a = StringFunction.modify(s, a -> a.toUpperCase());
```





Beispiel Konstruktorreferenz

- Im Vergleich Methodenreferenz / λ -Ausdruck

```
String result4 = StringFunction.modify(s, String::new);  
String result4b = StringFunction.modify(s, a -> new String(a));
```





Variablen im Scope eines λ -Ausdrucks

- Ein λ -Ausdruck definiert keinen eigenen Gültigkeitsbereich, sondern „sieht“ seine Umgebung genau wie der Code vor / nach dem λ -Ausdruck
- In einem λ -Ausdruck können daher keine (lokalen) Variablen deklariert werden, die einen Namen haben, der in dem umgebenden Scope bereits enthalten ist
- `this` bezieht sich bei einem λ -Ausdruck auf die Definitionsumgebung (ähnliches für `super`)
- Ein λ -Ausdruck kann Instanz- und Klassenvariablen der umgebenden Klasse lesen und schreiben (sofern Zugriff möglich)
- Auf lokale bzw. Parametervariable des umgebenden Gültigkeitsbereiches hat ein λ -Ausdruck nur Zugriff, wenn sie **effektiv final** sind (siehe später)



Variablen im Scope eines λ -Ausdrucks

```
public class InnerVsLambdaThis {  
    public InnerVsLambdaThis() {  
        int a = 8;  
        Runnable lambdaRun =  
            () -> System.out.println(this.getClass().getName() + " " + a);  
        Runnable innerRun = new Runnable() {  
            int a = 2;  
            @Override  
            public void run() {  
                System.out.println(this.getClass().getName() + " " + a);  
            }  
        };  
        lambdaRun.run(); // InnerVsLambdaThis 8  
        innerRun.run(); // InnerVsLambdaThis$1 2  
    }  
    public static void main(String[] args) { new InnerVsLambdaThis(); }  
}
```





Variablen im Scope eines λ -Ausdrucks

```
public class StringTesterQuiz {
    String sAttr = "Läuft was? ";

    public static void main(String[] args) {
        String s = "Geht das? ";
        String result;

        result = StringFunction.modify(s, x -> x + "?!?"); // 1
        result = StringFunction.modify(s, s -> s + "?!?"); // 2
        result = StringFunction.modify(s, t -> "Hey" + s); // 3
        result = StringFunction.modify(s, t -> t = "Nein!"); // 4
        result = StringFunction.modify(s, t -> s = "Nein!"); // 5
        result = StringFunction.modify(s, t -> { String r = " Ja!"; return (t + r); }); // 6
        result = StringFunction.modify(s, t -> { String s = " Ja!"; return (t + s); }); // 7
        StringTesterQuiz test1 = new StringTesterQuiz();
        result = StringFunction.modify(s, t -> test1.sAttr += "Immer!"); // 8
        result = test1.s_fun(); // 9
    }

    public String s_fun() {
        return StringFunction.modify(this.sAttr, sAttr -> this.sAttr.toUpperCase());
    }
}
```




Variablen im Scope eines λ -Ausdrucks

- Geben Sie für jede Zeile an, welchen Wert `result` hat, falls die Zeile korrekter Java-Code ist, oder warum die Zeile nicht kompiliert.



Variablen im Scope eines λ -Ausdrucks - Lösung

```
result = StringFunction.modify(s, x -> x + "?!?"); // Geht das? ?!?  
//result = StringFunction.modify(s, s -> s + "?!?"); // Compile Error: cannot redeclare ...  
result = StringFunction.modify(s, t -> "Hey" + s); // HeyGeht das?  
result = StringFunction.modify(s, t -> t = "Nein!"); // Nein!  
//result = StringFunction.modify(s, t -> s = "Nein!"); // Compile Error: must be final ...  
result = StringFunction.modify(s, t -> { String r = " Ja!"; return (t + r); }); // Geht das? Ja!  
//result = StringFunction.modify(s, t -> { String s = " Ja!"; return (t + s); });  
// Compile Error: cannot redeclare ...  
  
StringTesterQuiz test1 = new StringTesterQuiz();  
result = StringFunction.modify(s, t -> test1.sAttr += "Immer!"); // Läuft was? Immer!  
result = test1.s_fun(); // Läuft was? IMMER!
```





Effektiv finale lokale Variablen

- Ein λ -Ausdruck darf nur auf lokale Variablen des umgebenden Gültigkeitsbereichs zugreifen, wenn diese effektiv final sind, d.h. ihr Wert ändert sich nicht mehr nach der Initialisierung. Wir könnten sie also (ohne Compilerfehler) als final deklarieren.
- Warum?
 - Lokale Variablen liegen auf dem Stack. Bei nebenläufiger Ausführung hält jeder Thread seinen eigenen Stack. → Wird ein λ -Ausdruck der `run()`-Methode eines neuen Threads übergeben, können die Werte lokaler Variablen des umgebenden Scopes einmal mit übergeben werden. Der Stack des aufrufenden Thread kann und braucht danach nicht mehr beobachtet werden, da sich eine effektiv finale Variable nicht ändert.
- Aber:
 - Instanz-Variablen dürfen sich ändern, aber in einem λ -Ausdruck referenziert werden.
- Wieso?!
 - Sie liegen auf dem Heap → Wenn Threads nebenläufig auf dem Heap agieren, müssen sie sich ohnehin darum kümmern, dass es keine Konflikte gibt (s. Kapitel Threads).



Beispiel: Effektiv final

- Compiler registriert, wenn lokale Variablen modifiziert werden.

```
public void effectiveFinal() {  
    String s = "Hello";  
    String result = StringFunction.modify(s, t -> t = s + "World!");  
    s = "Changed";  
}
```



- Lösung

```
public void effectiveFinal() {  
    String s = "Hello";  
    String s2 = s;  
    String result = StringFunction.modify(s, t -> t = s2 + "World!");  
    s = "Changed";  
}
```





Programmieren

12. Lambdas und Streams

12.2 `java.util.function`



BuiltIn Functional Interfaces

- `java.util.function` stellt einige nützliche funktionale Interfaces bereit.
 - **Prädikate**: Prüfe ein/alle Objekt(en) auf die Eigenschaft p in einer Sammlung.
 - **Funktionen**: Anwendung einer / mehrerer Operation(en) auf ein Objekt.
 - **Binärer Operator**: Anwendung einer Operation auf 2 Operanden desselben Typs.
 - **Verbraucher**: Nimmt ein Inputargument entgegen und gibt kein Ergebnis zurück.
- Gerne in Kombination verwendet:
 - „Suche alle Objekte mit den Eigenschaften p1, . . . pn und führe dann die Operationen m1, . . . mk auf ihnen aus.“
- Beispiele:
 - `DoublePredicate`, `IntPredicate`, `LongPredicate`
 - Prädikat: in: double/int/long , out: boolean
 - `DoubleUnaryOperator`, `IntUnaryOperator`, `LongUnaryOperator`
 - Funktion: in: double/int/long , out: double/int/long
 - `DoubleBinaryOperator`, `IntBinaryOperator`, `LongBinaryOperator`
 - Binärer Operator: in: 2x double/int/long , out: 1x double/int/long
 - `DoubleConsumer`, `IntConsumer`, `LongConsumer`
 - Verbraucher (Consumer): in: double/int/long , out: void



Functional Interfaces: Auswahl

Interface	Lambda-Ausdruck	Auswertungsmethode	Bemerkung
Consumer<T>	<code>(T x) -> <void>;</code>	<code>accept(T x)</code>	ein Parameter, keine Rückgabe (z.B. durch Aufruf einer void-Methode)
Supplier<T>	<code>() -> const;</code>	<code>get()</code>	kein Parameter, Rückgabe vom Typ T
Predicate<T>	<code>x -> x < 5;</code>	<code>test(T x)</code>	ein Parameter vom Typ T, Rückgabe vom Typ boolean
Function<T,R>	<code>x -> x*x - x + 1;</code>	<code>apply(T x)</code>	ein Parameter vom Typ T, Rückgabe vom Typ R
BiFunction<T,U,R>	<code>(x,y) -> x*x - y*y;</code>	<code>apply(T x, U y)</code>	zwei Parameter vom Typ T und U, Rückgabe vom Typ R
UnaryOperator<T>	<code>x -> x*x - x + 1;</code>	<code>apply(T x)</code>	ein Parameter vom Typ T, Rückgabe vom Typ T
BinaryOperator<T>	<code>(x,y) -> x - y;</code>	<code>apply(T x, T y)</code>	zwei Parameter vom Typ T und Rückgabe vom Typ T

© Andreas de Vries: Funktionale Programmierung und Streams in Java



Functional Interface „Predicate“

```
@FunctionalInterface
public interface Predicate<T> {
    boolean test(T t);
}
```



- Idee:
 - Generische Schnittstelle, um Objekt(e) in einer Sammlung auf die Eigenschaft p zu testen
- Beispiel:

```
Predicate<Player> cheapTransfer =
    p -> p.getTransferCost() < 3_000_000;
if (cheapTransfer.test(somePlayer)) {
    // ...
}
```





Vergleich mit und ohne Predicate

- Ohne Nutzung von Predicate, zwei Anwendungen

```
// erzeugt Liste aller Spieler, deren Gehaltsvorstellungen unter limit liegen
public static List<Player> findAllPlayersBySalary(List<Player> players, double limit) {
    ArrayList<Player> results = new ArrayList<Player>();
    for (Player p : players) {
        if (p.getRequiredSalaryPerYear() < limit) { results.add(p); }
    }
    return results;
}
```



```
// erzeugt Liste aller Integer, die durch div teilbar sind
public static List<Integer> findAllMultiples(List<Integer> ints, int div) {
    ArrayList<Integer> results = new ArrayList<Integer>();
    for (Integer i : ints) {
        if (i % div == 0) { results.add(i); }
    }
    return results;
}
```





Vergleich mit und ohne Predicate

```
// Nutzung irgendwo anders  
List<Player> candidates = findAllPlayersBySalary(players, 3_000_000);  
List<Integer> intsDiv3 = findAllMultiples(ints, 3);
```



- Kritik:
 - Codewiederholung
 - Trennung der Prädikatdefinition und der Nutzung zum Selektieren → schlecht lesbar
 - in den Prüfmethode ist die Prüfbedingung zwischen Parameter und Schleifenrumpf verteilt → schlecht lesbar
 - nur aufwendig erweiterbar: Was, wenn wir alle durch `div` teilbaren Zahlen suchen, die größer als `min` sind? → noch schlechter lesbarer Code



Vergleich mit und ohne Predicate

- Code mit dem Functional Interface Predicate und generischen Typen

```
public static <T> List<T> allMatches(List<T> candidates, Predicate<T> pred) {  
    ArrayList<T> results = new ArrayList<T>();  
    for (T possibleMatch : candidates) {  
        if (pred.test(possibleMatch)) { results.add(possibleMatch); }  
    }  
    return results;  
}
```



```
// Nutzung irgendwo anders mit Lambdas  
List<Player> potentialPlayers = allMatches(players,  
                                           p -> p.getRequiredSalaryPerYear() < 3000000);  
  
List<Integer> intsDiv13 = allMatches(ints, i -> i % 3 == 0);  
  
List<Country> bigCountries = allMatches(countries,  
                                       c -> c.getPopulation() > 50_000_000);
```





Vergleich mit und ohne Predicate

- Verbesserung:
 - Eine Hilfsmethode `allMatches` für alle Filter auf Listen
 - Das Prädikat steht als λ -Ausdruck direkt bei der Nutzung.
 - Kombination mit weiteren Prädikaten leicht möglich
 - im λ -Ausdruck mit logischen Operatoren, z.B. `&&`, `||`, `!`

```
List<Integer> intsDiv14 = allMatches(ints, i -> i % 3 == 0 && i > 5);
```



- über die Default-Methoden von `Predicate`: z.B. `and`, `or`, `negate`

```
Predicate<Player> filter = p -> p.getRequiredSalaryPerYear() > 3e6;  
filter = filter.and( p -> p.getTransferCost() < 1000000 );  
List<Player> filteredList = allMatches( players, filter );
```



```
Predicate<Integer> isEven = i -> i % 2 == 0;  
Predicate<Integer> isOdd = isEven.negate();
```





Functional Interface „Function“

```
@FunctionalInterface
public interface Function<T,R> {
    R apply(T t);
}
```



```
@FunctionalInterface
public interface BiFunction<T,U,R> {
    R apply(T t, U u);
}
```



- Idee:
 - Generische Schnittstelle, um Funktion $f : T \rightarrow R$ zu beschreiben.
 - Die Funktion f wird auf einen Wert / eine Sammlung von Werten angewendet.
 - Analog beschreibt `BiFunction` eine zweistellige Funktion: $g : T \times U \rightarrow R$
- Beispiel:

```
Function<Player, Double> payBonus =
    p -> p.getSalary() + 100_000;
for (Player p : players ) {
    p.setSalary(payBonus.apply(p));
}
```





Vergleich mit und ohne Function

- Ohne Nutzung von Function, zwei Anwendungen

```
public static double salaryAverage(List<Player> players) {  
    double sum = 0.0;  
    for (Player p : players) { sum += p.getSalary(); }  
    return (sum / players.size());  
}
```



```
public static double calcAverage(List<Integer> ints) {  
    double sum = 0.0;  
    for (Integer i : ints) { sum += i; }  
    return (sum / ints.size());  
}
```





Vergleich mit und ohne Function

- Ohne Nutzung von Function

```
// Nutzung irgendwo anders  
double salaryCost = salaryAverage(players);  
double mean = calcAverage(ints);
```



- Kritik (wie bei Predicate):
 - Codewiederholung
 - schlecht lesbar und schlecht erweiterbar



Vergleich mit und ohne Function

- Code mit dem Functional Interface `Function` und generischen Typen:

```
public static <T> double mapAverage(List<T> entries,  
                                   Function<T, Double> valProvider) {  
    double sum = 0.0;  
    for (T entry : entries) { sum += valProvider.apply(entry); }  
    return (sum / entries.size());  
}
```



```
// Nutzung irgendwo anders mit Methodenreferenz  
double avSalary = mapAverage(players, Player::getSalary);  
double mean2 = mapAverage(ints, Integer::doubleValue);  
  
ArrayList<Double> dlist =  
    new ArrayList<>(Arrays.asList(1.0, 2.0, 3.0, 4.0));  
double average = mapAverage(dlist, Function.identity());
```





Vergleich mit und ohne Function

- Verbesserung:
 - Eine Hilfsmethode `mapAverage` für jede Durchschnittsberechnung auf Listen
 - Die Funktion steht als λ -Ausdruck/Methodenreferenz direkt bei der Nutzung.
 - Kombination mit weiteren Funktionen leicht möglich:
 - im λ -Ausdruck

```
double avSalary2 = mapAverage(players, p -> p.getSalary()*2);
```



- und über die Default-Methoden von `Function`: z.B. `identity`, `andThen`, `compose` (Beispiel siehe unter „Higher Order Functions“ im nächsten Abschnitt)



Functional Interface „Binary Operator“

```
@FunctionalInterface
public interface BinaryOperator<T> {
    T apply(T t1, T t2);
}
```



- Idee
 - Generische Schnittstelle, um Funktion $f : T \times T \rightarrow T$ zu beschreiben.
 - Alle Parameter und das Ergebnis sind vom gleichen Typ.
 - Abgeleitet von `BiFunction<T, T, T>`.
- Beispiel:

```
BinaryOperator<Integer> adder = (n1, n2) -> n1 + n2;
```





Higher Order Functions

- Zentrale Idee des λ -Kalküls: Funktionen sind Werte
- In Java: Ein λ -Ausdruck kann als Parameter (Wert) übergeben werden
- Konsequenz: Methoden können auch einen λ -Ausdruck zurückgeben
- Nutzen:
 - In Java kann der Ergebnistyp einer Methode ein Interface sein.
 - Wenn man sich Funktionen als Werte vorstellt, kann man auch Funktionen höherer Ordnung haben, die
 - eine Funktionskomposition bilden: $f \circ g$
 - Prädikate negieren, oder mit and verknüpfen
 - eine Funktion n-mal hintereinander anwenden
 - eine ganze Liste von Funktionen hintereinander anwenden.



Higher Order Functions Beispiel

- Beispiel für die Verkettung von Funktionen (auf den folgenden Folien)
- `transform` wendet eine Funktion auf eine Liste von T-Objekten an

```
public static <T, R> List<R> transform(  
    List<T> origValues, Function<T, R> transformer) {  
    List<R> transformedValues = new ArrayList<>();  
    for (T value : origValues) {  
        transformedValues.add(transformer.apply(value));  
    }  
    return transformedValues;  
}
```





Higher Order Functions Beispiel

```
List<String> words = new ArrayList<>(  
    Arrays.asList("hallo", "hej", "hello", "hola", "salut", "ciao"));  
  
// Funktionsdefinition (1) mit Methodenreferenz  
Function<String, String> makeUpperCase = String::toUpperCase;  
// Anwendung der Funktion (1) auf Liste von Strings  
List<String> upperCaseWords = transform(words, makeUpperCase);  
// Ausgabe  
System.out.printf("Uppercase: %s.%n", upperCaseWords);  
  
// Funktionsdefinition (2) als λ-Ausdruck, da die zugrundeliegende Funktion  
// keinen Namen hat  
Function<String, String> makeExciting = word -> word + ": yeah!";  
// Anwendung der Funktion (2) auf Liste von Strings  
List<String> excitingWords = transform(words, makeExciting);  
// Ausgabe  
System.out.printf("Exciting:%s.%n", excitingWords);
```



Higher Order Functions Beispiel

```
// Funktionsdefinition (3) als Verkettung von Funktionen
Function<String, String> upperCaseBeforeExciting =
    makeExciting.compose(makeUpperCase);

// Anwendung der Funktion (3) auf Liste von Strings
List<String> upperCaseBeforeExcitingWords =
    transform(words, makeUpperCaseBeforeExciting);

// Ausgabe
System.out.printf("Upper then Exciting: %s.%n", upperCaseBeforeExcitingWords);

// Funktionsdefinition (4) als Verkettung von Funktionen
Function<String, String> makeUpperCaseAfterExciting =
    makeExciting.andThen(makeUpperCase);

// Anwendung der Funktion (4) auf Liste von Strings
List<String> upperCaseAfterExcitingWords = transform(words,
    upperCaseAfterExciting);

// Ausgabe
System.out.printf("Exciting then Upper: %s.%n", upperCaseAfterExcitingWords);
```



Higher Order Functions Beispiel

- Ausgabe

Uppercase: [HALLO, HEJ, HELLO, HOLA, SALUT, CIAO].

Exciting:[hallo: yeah!, hej: yeah!, hello: yeah!, hola: yeah!, salut: yeah!, ciao: yeah!].

Upper then Exciting: [HALLO: yeah!, HEJ: yeah!, HELLO: yeah!, HOLA: yeah!, SALUT: yeah!, CIAO: yeah!].

Exciting thrn Upper: [HALLO: YEAH!, HEJ: YEAH!, HELLO: YEAH!, HOLA: YEAH!, SALUT: YEAH!, CIAO: YEAH!].



Built-in-Operationen auf Listen und Maps

- Möglichkeiten zur Realisierung von Built-in-Operationen auf Listen und Maps:
- Streams benutzen (arbeitet auf Streams) → nächster Abschnitt
- Higher-Order-Built-in-Listenfunktionen nutzen (arbeitet auf List / Map), aber: erst die Definition lesen und verstehen
- Für Listen:
 - `forEach` - für alle Iterables nutzbar
 - `removeIf` - für alle Collections nutzbar
 - `removeAll`
 - `sort`
- Für Maps:
 - `forEach`
 - `computeIfAbsent` - auch `compute`, `computeIfPresent`
 - `merge`
 - `replaceAll`



Built-in-Operationen auf Listen und Maps

- Beispiel

```
// Achtung: Liste wird verändert!  
ArrayList<Integer> i_list = new ArrayList<>(  
    Arrays.asList(1, 2, 3, 4, 5, 6, 7, 8, 9));  
i_list.removeIf(e -> e % 3 != 0);  
  
ArrayList<Player> potentialPlayers = null;  
// Achtung: auch keine Verkettung wegen Rückgabetyt  
potentialPlayers.removeIf(p -> p.getRequiredSalaryPerYear() > 3_000_000);  
potentialPlayers.sort((p1, p2) ->  
    p1.getFirstname().compareTo(p2.getFirstname()));
```



- Bewertung: Kompakte Lösung, falls die Liste modifiziert werden kann und die Built-in-Funktion genau passt.



Programmieren

12. Lambdas und Streams

12.3 Streams



Motivation Ströme

- Ein **Stream (Datenstrom)** ist eine Folge gleichstrukturierter Elemente, deren Ende nicht im Voraus festgelegt ist.
 - Stream = $x_1, x_2, x_3, x_4, x_5, \dots$
- Idee:
 - Eine **wiederkehrende Arbeitsfolge** (z.B. **Filter oder Operationen**) soll auf eine Sammlung von Objekten
 - in **passender Reihenfolge** angewendet
 - und das **Ergebnis dann geeignet abgelegt** werden.
- Beispiel:
 - Aus einer Liste von Wörtern
 - filtern: alle Wörter mit 5 Buchstaben, die ein e enthalten,
 - Operationen anwenden: alphabetisch sortieren, `toUpperCase`
 - Ergebnis ablegen: alles in eine Datei schreiben.



Streams: Kernkonzepte

- Streams dienen dazu, (eine große Menge von) Daten von einer Quelle (Datenstruktur, I/O-Ressource, . . .) durch eine Abfolge (Pipeline) von Operationen zu verarbeiten und abschließend zu aggregieren oder einer Datensenke zuzuführen.
- Streams bieten dazu eine Reihe sehr nützlicher Verarbeitungsmethoden (Higher-Order-Functions), die über Listen hinaus gehen:
 - `forEach`, `filter`, `map`, `reduce`, `min`, `distinct`, `sorted`, `limit` ...
- Streams haben bessere Eigenschaften als Listen: Streams
 - sind ausdrucksmächtiger Listen,
 - bieten Lazy Evaluation,
 - bieten automatische Parallelisierung
 - und unendliche Ströme = Folge von Elementen, deren Ende nicht bereits zu Beginn definiert sein muss.



Streams: Kernkonzepte

- Streams **speichern keine Daten**, sondern sind nur Wrapper um existierende Datenstrukturen. Streams ändern die zugrundeliegende Datenstruktur nicht.
- Streams können **sequentiell** oder **parallel** ausgewertet werden. Es ist – im Unterschied zu Collections oder Arrays – allerdings nicht möglich, gezielt auf einzelne Elemente in einem Stream zuzugreifen.
- Streams erlauben, jede Menge Boilerplate-Code (d.h. Code, der in fast gleicher Form an vielen Stellen eingefügt werden muss) einzusparen, insbesondere, da λ -Ausdrücke genutzt werden können.



Anmerkung: Streams für I/O

- Das Konzept Streams kennen Sie bereits vom Lesen und Schreiben von Dateien:

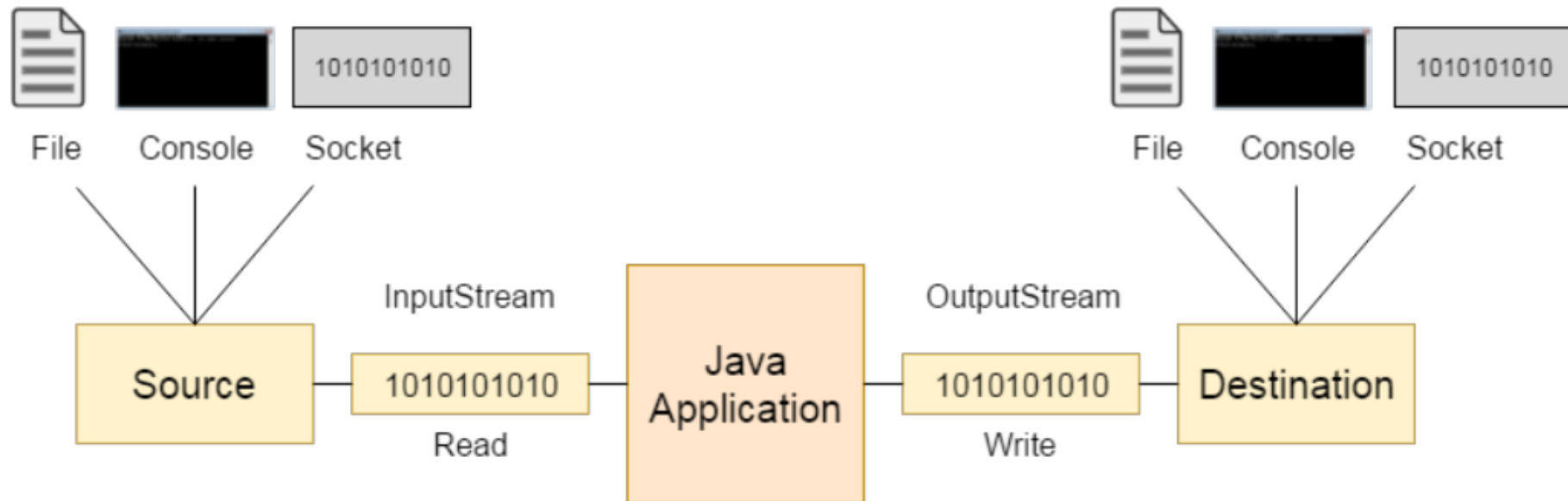
```
File file = new File("file.txt") ;  
FileReader fileReader = new FileReader(file);  
BufferedReader bufferedReader = new BufferedReader(fileReader);  
bufferedReader.readLine();
```



- Dabei wird ein Datenstrom aus einer Datei gelesen mit `FileReader`.
- `BufferedReader` ergänzt nützliche Eigenschaften.
- Schreiben in Dateien: analog mit `BufferedWriter`, `FileWriter`.
- Gleiches Konzept, aber angewendet auf allgemeinere Datenverarbeitung.

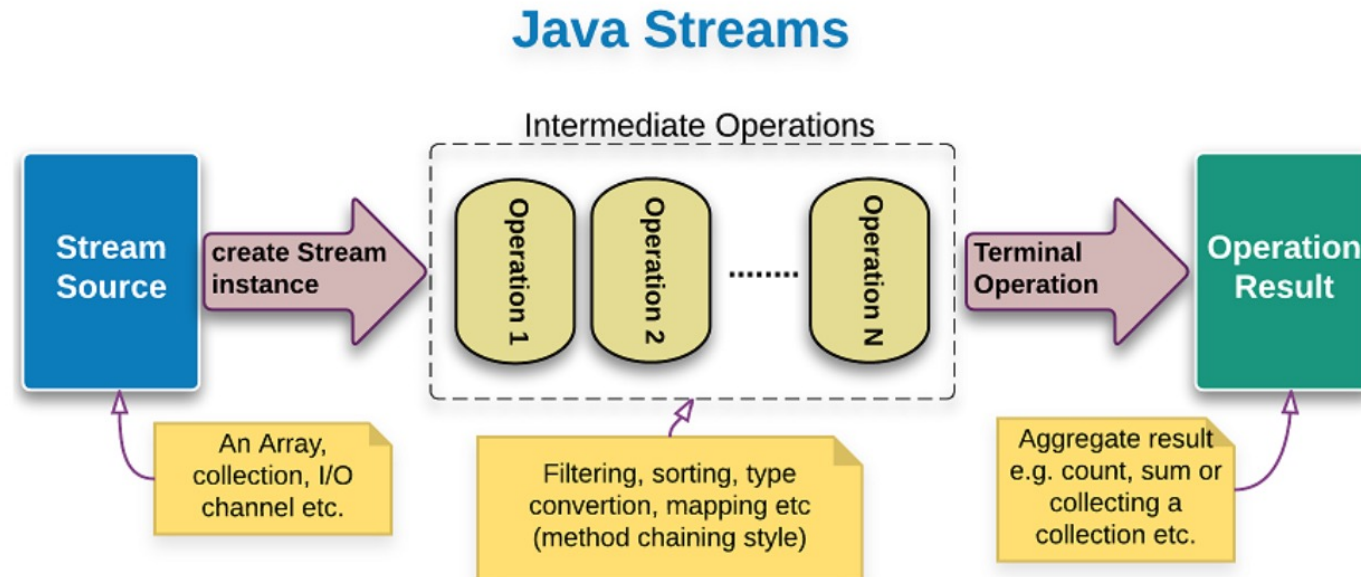


Anmerkung: Streams für I/O



© Java64bit.com

Streams: Verarbeitungsprozess



- **Intermediäre Operation:**
 - erwartet einen Stream und gibt einen Stream zurück,
 - ermöglicht Weiterverarbeitung in einer „Pipeline“
- **Terminale Operation:**
 - gibt anderen Ergebnistyp zurück; beendet Verarbeitung



Word-Processing als Beispiel für Streams



```
try (Stream<String> w_stream
    = Files.lines(Paths.get(inputFile))) {
    List<String> wordlist = w_stream
        .flatMap(string -> Stream.of(string.split(" ")))
        // Zeile -> Wörter
        .filter(s -> s.length() > 6) // filtern: nur lange Wörter
        .map(String::toLowerCase) // nur Kleinbuchstaben
        .distinct() // keine Wiederholungen
        .sorted() // (alphabetisch) sortieren
        .collect(Collectors.toList()); // Ergebnis als Liste speichern

    // Liste in Datei schreiben
    Files.write(Paths.get(outputFile), wordlist, Charset.defaultCharset());
} catch (IOException e) { e.printStackTrace(); }
```



Streams erzeugen

- Gängige Möglichkeiten

```
// Collection-basierter Stream
List<String> textlist = List.of("Dunke", "wars", "der", "Mond");
Stream<String> seq1 = textlist.stream();
Stream<String> par1 = textlist.parallelStream();

// Streams aus Arrays
String[] textarr = {"Dunkel", "wars", "der", "Mond"};
Stream<String> seq2 = Arrays.stream(textarr);

// Stream-Generatoren (unendlicher Strom, falls nicht z.B. mit limit begrenzt):
Stream<Double> random = Stream.generate(Math::random);
Stream<Integer> even = Stream.iterate(2, n -> n + 2);

// aus einzelnen Datenelementen
Player p1 = new Player(), p2 = new Player(), p3 = new Player();
Stream<Player> seq3 = Stream.of(p1, p2, p3);
```



- Für primitive Datentypen gibt es noch Streams, die speziell für diese optimiert sind (z.B. `IntStream`, `LongStream` und `DoubleStream`).



Intermediäre Stream-Operationen: State

- **Intermediäre** Operationen: Erwarten einen Strom und geben einen Strom zurück
- Einteilung in „stateful“ (**zustandsbehaftet**) bzw. „stateless“ (**zustandslos**)
- **Stateless**
 - Ein Strom wird Element für Element verarbeitet; für jedes Element wird – unabhängig von anderen Elementen – eine Operation ausgeführt.
 - Stateless-Operationen benötigen daher auch bei langen Strömen keinen zusätzlichen Speicher.
- **Stateful**
 - Manche intermediäre Operationen müssen den gesamten Strom lesen, um ihre Aufgabe erfüllen zu können (z.B. den Strom zu sortieren) oder sich Daten über bereits gelesene Elemente merken (z.B. jedes Element nur einmalig ausgeben, `distinct`).
 - Dadurch sind intermediäre Operationen mit Zustand („stateful“) bei der Verarbeitung von langen Strömen durch ihren Speicherverbrauch ein Risiko.



Intermediäre Stream-Operationen: Übersicht

- Die wichtigsten intermediären Operationen für Streams sind:
 - `sorted (stateful)`
 - Liefert einen nach der natürlichen Ordnung bzw. einem übergebenen `Comparator` sortierten Strom
 - `limit`
 - Liefert einen auf eine Anzahl von Elementen begrenzten Strom
 - `skip`
 - Liefert einen Strom, bei dem die ersten Elemente übersprungen werden
 - `filter`
 - Liefert einen Strom, der nur solche Elemente enthält, für die ein Prädikat gilt
 - `distinct (stateful)`
 - Liefert einen Strom, in dem jedes Element nur einmalig (`equals`) vorkommt
 - `map / flatMap`
 - Liefert einen Strom, bei dem die der Eingabestroms mit `map` auf den Ausgabestroms abgebildet werden (`flatMap`: „flache“ Ausgabe)
 - `peek(Consumer)`
 - Liefert einen Strom, bei dem für jedes Element der `Consumer` ausgeführt wird (der Inhalt des Stroms wird nicht verändert)
- Alle Operatoren können mit Lambda-Ausdrücken oder mit Methodenreferenzen arbeiten.



Terminale Stream-Operationen: Übersicht

- Die wichtigsten terminalen Operationen für Streams sind:
 - `forEach(Consumer)`
 - Führt den übergebenen Consumer für jedes Element des Stroms aus
 - `findFirst` / `findAny`
 - Bricht die Verarbeitung ab, sobald das erste / irgendein Element gefunden wird; dieses wird als `Optional`-Objekt zurückgegeben
 - `anyMatch` / `allMatch` / `noneMatch`
 - Gibt booleschen Wert zurück, ob irgendein Element des Stroms / alle Elemente / kein Element zum übergebenen Prädikat passt bzw. passen
 - `reduce`
 - Reduziert den Strom auf einen Ergebniswert (übergebene Funktion mit Akkumulator)
 - `count`
 - Liefert die Anzahl Elemente in einem Stream
 - `min` / `max`
 - Liefert das minimale / maximale Element im Strom gemäß dem spezifizierten Comparator
 - `toArray`
 - Konvertiert einen Strom in ein Array
 - `collect`
 - Konvertiert einen Strom in einen anderen Container, z.B. eine Liste, eine Menge usw.



... zurück zu den üblichen Datenstrukturen

- zu Listen oder anderen Collection-Typen
 - `someStream.collect(Collectors.toList())`
- zu einem Array von Objekten
 - `someStream.toArray(EntryType[]::new)`
- Hinweis: Beide Methoden sind terminale Operationen, die erst am Ende der Verarbeitungskette angewendet werden:
- ```
List<SomeType> values = someStream
 .map(...)
 .filter(...)
 .map(...)
 .filter(...)
 .collect(...);
```



# Beispiel: Operationen auf Integer ArrayList

```
// for-Schleife mit explizitem Iterator
ArrayList<Integer> ilist = new ArrayList<>(Arrays.asList(3, 1, 8, 4, 0, 3, 5, 2));
System.out.println(ilist); // [3, 1, 8, 4, 0, 3, 5, 2]
Collections.sort(ilist, (i1, i2) -> i1.compareTo(i2));
System.out.println(ilist); // [0, 1, 2, 3, 3, 4, 5, 8]
for (Iterator<Integer> it = ilist.iterator(); it.hasNext();) {
 Integer i = it.next();
 if (i % 3 == 0) { it.remove(); }
}
System.out.println(ilist); // [1, 2, 4, 5, 8]
```



```
// Beispiel mit Built-in-Methoden von ArrayList
ArrayList<Integer> ilist2 = new ArrayList<>(Arrays.asList(3, 1, 8, 4, 0, 3, 5, 2));
ilist2.forEach(i -> System.out.print(i + " ")); // 3 1 8 4 0 3 5 2
System.out.println();
ilist2.sort((i1, i2) -> i1.compareTo(i2));
ilist2.forEach(i -> System.out.print(i + " ")); // 0 1 2 3 3 4 5 8
System.out.println();
ilist2.removeIf(e -> e % 3 == 0);
ilist2.forEach(i -> System.out.print(i + " ")); // 1 2 4 5 8
```





# Beispiel: Operationen auf Integer ArrayList

```
// Built-in-Methoden von Streams, ohne Speicherung des Ergebnisses
ArrayList<Integer> ildist3 = new ArrayList<>(Arrays.asList(3, 1, 8, 4, 0, 3, 5, 2));
ildist3.stream()
 .sorted((i1, i2) -> i1.compareTo(i2))
 .filter(e1 -> e1 % 3 != 0)
 .forEach(p -> System.out.print(p.toString() + " ")); // 1 2 4 5 8
// forEach als terminale Operation; wegen + " " keine Methodenreferenz, sondern
// Lambda, Ausgabe von Zwischenergebnissen hier nicht so einfach
```



```
// Built-in-Methoden von Streams, mit Speicherung des Ergebnisses
ArrayList<Integer> ildist4 = new ArrayList<>(Arrays.asList(3, 1, 8, 4, 0, 3, 5, 2));
List<Integer> result = ildist4.stream()
 .sorted((i1, i2) -> i1.compareTo(i2))
 .filter(e1 -> e1 % 3 != 0)
 .collect(Collectors.toList());
System.out.println(result); // [1, 2, 4, 5, 8]
```







# Weiteres Material

- Stream Tutorial von Marty Hall: Marty Hall,
- coreservlets: Java 8 Lambdas and Streams
- Java Language Specification:
  - <https://docs.oracle.com/javase/specs/jls/se8/html/jls-15.html#jls-15.27>